

✓ NLP class assignment – BBC Text Dataset

- **Q1:** Preprocessing pipeline: tokenize, lemmatize, remove stopwords
- **Q2:** TF-IDF vectors and top-5 terms for 3 documents
- **Q3:** Load GloVe and find top-3 nearest words
- **Q4:** Solve the word analogy: `Queen - king + Man = ?`

Upload `bbc-text.csv` when asked, or place it in the same Colab folder.

✓ Install and import required libraries

```
# Install gensim for loading GloVe word embeddings
# Colab may already have some libraries, but this makes the notebook safer to run.
!pip install gensim -q

# Import basic libraries
import os
import re
import numpy as np
import pandas as pd

# Import NLP libraries
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

# Import TF-IDF tool
from sklearn.feature_extraction.text import TfidfVectorizer

# Import gensim downloader for GloVe
import gensim.downloader as api

# Download NLTK resources
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

27.9/27.9 MB 35.2 MB/s eta 0:00:00

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
True
```

✓ Load the CSV dataset

```
# If the CSV file is not found, Colab will ask you to upload it
if not os.path.exists('bbc-text.csv'):
    from google.colab import files
    uploaded = files.upload()

# Load the dataset
df = pd.read_csv('bbc-text.csv')

# Show first 5 rows
df.head()
```

Choose files `bbc-text.csv`
bbc-text.csv(text/csv) - 5057493 bytes, last modified: 25/04/2026 - 100% done
 Saving bbc-text.csv to bbc-text.csv

	category	text	
0	tech	tv future in the hands of viewers with home th...	
1	business	worldcom boss left books alone former worldc...	
2	sport	tigers wary of farrell gamble leicester say ...	
3	sport	yeading face newcastle in fa cup premiership s...	
4	entertainment	ocean s twelve raids box office ocean s twelve...	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Check column names
print(df.columns)

# Use the 'text' column if it exists, otherwise use the last column
text_column = 'text' if 'text' in df.columns else df.columns[-1]

# Keep only text values and remove empty rows
texts = df[text_column].dropna().astype(str).tolist()

print("Number of documents:", len(texts))
print("Example text:")
print(texts[0][:300])
```

```
Index(['category', 'text'], dtype='object')
Number of documents: 2225
Example text:
tv future in the hands of viewers with home theatre systems plasma high-definition tvs and digital video recorders moving into the living room the w
```

✓ Q1: Preprocessing pipeline

Steps used:

1. Convert text to lowercase
2. Tokenize using a simple regular expression
3. Remove stopwords
4. Lemmatize words

```
# Create stopword list and lemmatizer
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    # 1. Convert text to lowercase
    text = text.lower()

    # 2. Tokenize: keep only alphabetic words
    tokens = re.findall(r'\b[a-z]+\b', text)

    # 3. Remove stopwords and very short words
    tokens = [word for word in tokens if word not in stop_words and len(word) > 2]

    # 4. Lemmatize each word
    tokens = [lemmatizer.lemmatize(word) for word in tokens]

    return tokens

# Apply preprocessing to first document as an example
sample_tokens = preprocess_text(texts[0])

print("Original text:")
print(texts[0][:300])

print("\nProcessed tokens:")
print(sample_tokens[:50])
```

```
Original text:
tv future in the hands of viewers with home theatre systems plasma high-definition tvs and digital video recorders moving into the living room the w

Processed tokens:
['future', 'hand', 'viewer', 'home', 'theatre', 'system', 'plasma', 'high', 'definition', 'tv', 'digital', 'video', 'recorder', 'moving', 'living', 'room', 'way', 'people', 'w
```

```
# Preprocess all documents
processed_tokens = [preprocess_text(text) for text in texts]

# Join tokens back into strings because TF-IDF needs text input
processed_texts = [' '.join(tokens) for tokens in processed_tokens]

print("First processed document:")
print(processed_texts[0][:300])
```

```
First processed document:
future hand viewer home theatre system plasma high definition tv digital video recorder moving living room way people watch radically different five y
```

Q2: Generate TF-IDF vectors and list top-5 terms for 3 documents

```
# Create TF-IDF vectorizer
tfidf = TfidfVectorizer()

# Generate TF-IDF matrix
tfidf_matrix = tfidf.fit_transform(processed_texts)

# Get all feature names / terms
terms = tfidf.get_feature_names_out()

print("TF-IDF matrix shape:", tfidf_matrix.shape)
```

TF-IDF matrix shape: (2225, 24553)

```
def get_top_terms_for_document(doc_index, top_n=5):
    # Get TF-IDF scores for one document
    row = tfidf_matrix[doc_index].toarray().flatten()

    # Get indexes of top scores
    top_indexes = row.argsort()[::-1][:top_n]

    # Return term and score pairs
    return [(terms[i], row[i]) for i in top_indexes]

# Show top-5 terms for first 3 documents
for i in range(3):
    print(f"\nDocument {i+1} top 5 TF-IDF terms:")
    top_terms = get_top_terms_for_document(i, top_n=5)

    for term, score in top_terms:
        print(term, ":", round(score, 4))
```

Document 1 top 5 TF-IDF terms:

dvr : 0.2497
brand : 0.2247
hanlon : 0.1972
channel : 0.1779
definition : 0.1623

Document 2 top 5 TF-IDF terms:

worldcom : 0.4655
ebbers : 0.4271
myers : 0.339
accounting : 0.2653
weingarten : 0.1436

Document 3 top 5 TF-IDF terms:

farrell : 0.5089
gamble : 0.3004
leicester : 0.2309
rugby : 0.2041
league : 0.1871

Q3: Load GloVe and find top-3 nearest words

Using small pretrained model `glove-wiki-gigaword-50` to keep it simple.

```
# Load small GloVe model
# This may take a little time the first time it runs.
glove_model = api.load('glove-wiki-gigaword-50')

print("GloVe model loaded successfully!")
```

[=====] 100.0% 66.0/66.0MB downloaded
GloVe model loaded successfully!

```
# Words to be queried
query_words = ['election', 'smartphone', 'mountain']

# Find top-3 nearest words for each query word
for word in query_words:
    print(f"\nTop 3 nearest words to '{word}':")
```

```
# Check if word exists in GloVe vocabulary
if word in glove_model:
    similar_words = glove_model.most_similar(word, topn=3)

    for similar_word, score in similar_words:
        print(similar_word, ":", round(score, 4))
else:
    print("Word not found in GloVe vocabulary.")
```

Top 3 nearest words to 'election':
 elections : 0.9583
 polls : 0.8905
 vote : 0.8788

Top 3 nearest words to 'smartphone':
 iphone : 0.9002
 smartphones : 0.8813
 ipad : 0.877

Top 3 nearest words to 'mountain':
 mountains : 0.8784
 valley : 0.8215
 hills : 0.8039

Q4: Word analogy

Question: Queen - king + Man = ?

In word vectors, this is calculated as:

queen + man - king

```
# Solve analogy: queen - king + man
# In gensim, positive words are added and negative words are subtracted.
analogy_result = glove_model.most_similar(
    positive=['queen', 'man'],
    negative=['king'],
    topn=5
)

print("Queen - king + Man = ?")
print("\nTop 5 results:")

for word, score in analogy_result:
    print(word, ":", round(score, 4))

print("\nBest answer:", analogy_result[0][0])
```

Queen - king + Man = ?

Top 5 results:
 woman : 0.8947
 girl : 0.849
 her : 0.7932
 boy : 0.7824
 she : 0.7701

Best answer: woman

Note

The BBC CSV is used for **Q1 and Q2** because it contains text documents.

For **Q3 and Q4**, we use **GloVe pretrained word vectors** because word analogy questions require numerical word embeddings trained on a very large corpus.

